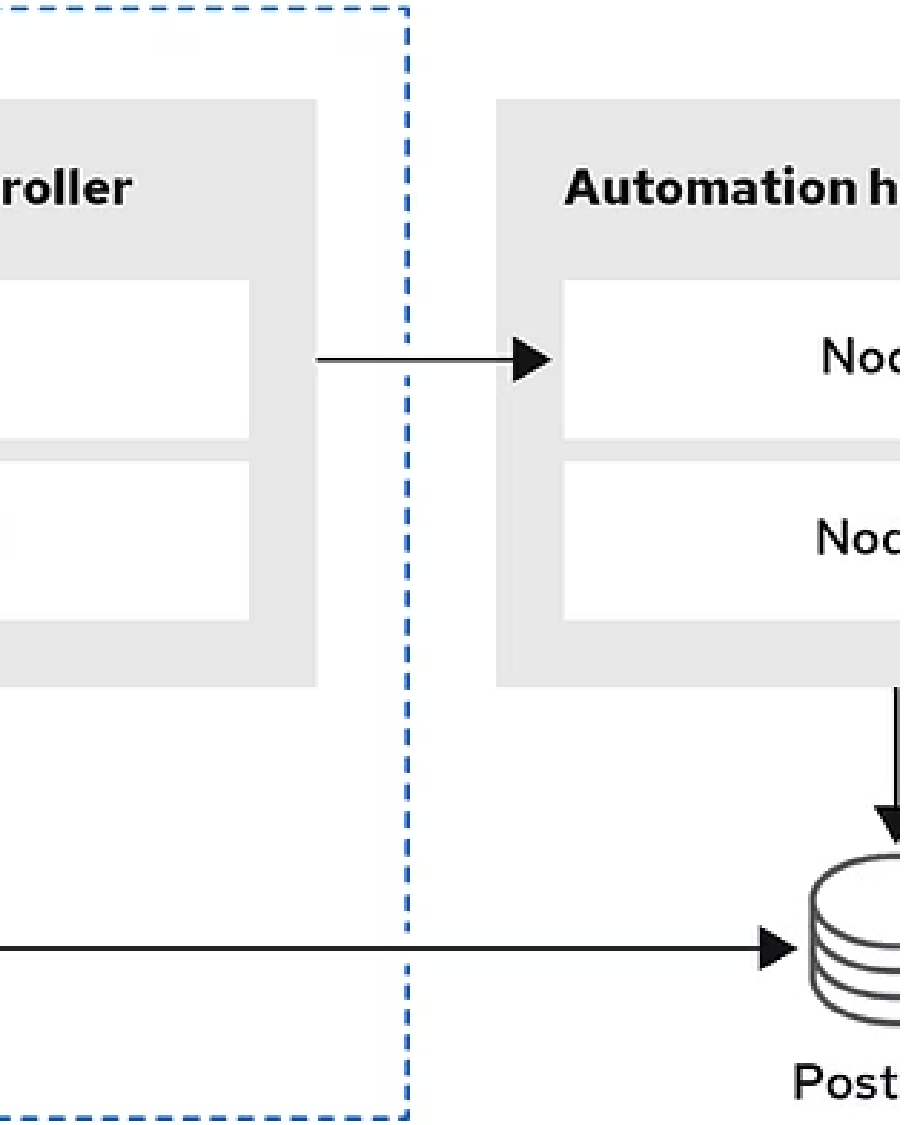
Node Architecture in Ansible Automation Platform

A trainer-ready, enterprise-grade guide for IT administrators and engineers new to AAP — explaining what each node type is, why it exists, and how it behaves in real production environments.



Understanding Node Architecture in AAP

In Red Hat Ansible Automation Platform, **nodes** are the servers — VMs or physical machines — that collectively run, manage, and execute automation jobs. Think of AAP like a factory:

🗓️ Plan & Schedule

Control nodes organize and dispatch work across the platform

⚙️ Execute

Execution nodes do the actual automation heavy lifting

🔗 Connect

Hop nodes bridge remote and isolated network zones

👥 Organize

Instance groups cluster workers by environment or purpose

What Are Nodes in AAP?

A **node** is any system that participates in the automation lifecycle — from accepting requests to executing playbooks and relaying traffic across networks.

A node can:

- Accept and schedule job requests
- Execute Ansible playbooks
- Communicate with other nodes
- Connect to isolated or remote networks

The four node types:

Control Nodes

Brain of the platform

Execution Nodes

Muscle of the platform

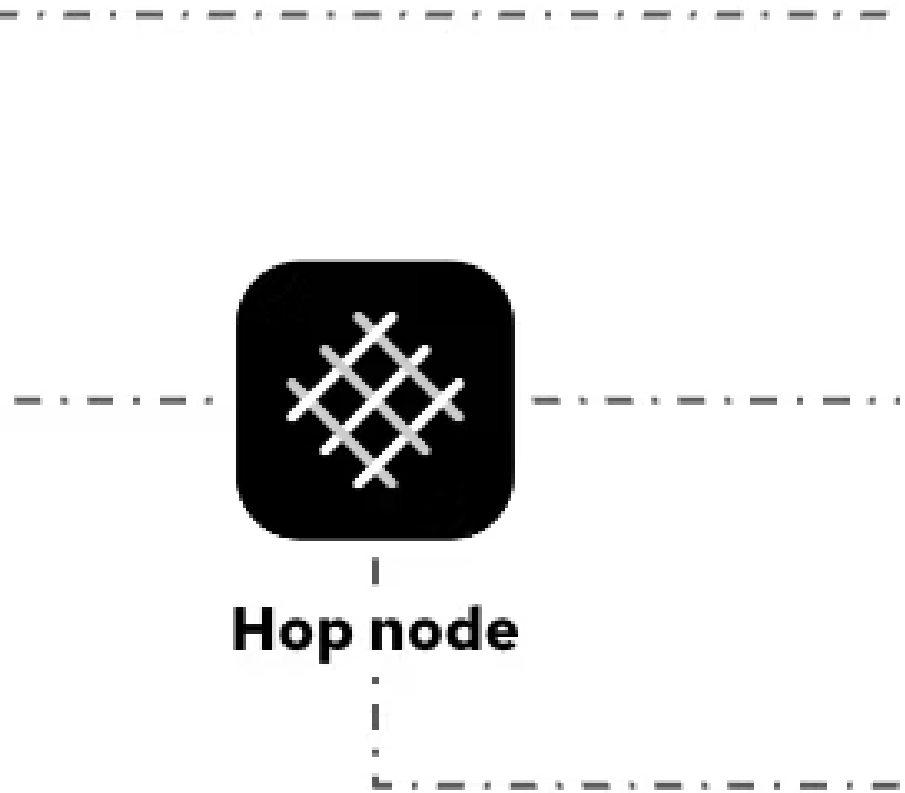
Hybrid Nodes

Control + execution combined

Hop Nodes

Network bridges

Each node type exists for a reason: **scalability, security, and reliability** in enterprise automation environments.



NODE TYPES

Control Nodes — The Brain of the Platform

What They Do

- Host the web UI and expose REST APIs
- Handle RBAC and authentication
- Schedule jobs and route them to execution nodes
- Resolve inventories and inject credentials

What They Do NOT Do

In modern AAP, control nodes **do not execute playbooks**. That work is fully delegated to execution nodes — keeping the control plane lean and secure.

> 🛫 **Analogy:** Control Node = Air Traffic Control Tower — it directs all traffic but never flies the plane.

NODE TYPES

Execution Nodes — The Muscle of the Platform

What They Do

- Run Ansible playbooks against managed hosts
- Execute modules, roles, and collections
- Handle SSH, WinRM, and API connections

Why They Exist

- Offload heavy automation work from control nodes
- Scale independently from the UI and API layer
- Improve security with limited network access

🏭 **Analogy:** Execution Node = Factory Worker — it receives instructions and does the actual work.

NODE TYPES

Hybrid Nodes — For Labs & Small Environments

A hybrid node acts as **both a control and execution node** on a single machine — running the UI, API, scheduling, *and* playbook execution together.

✅ When to Use

- **Labs**
- **POCs**
- **Small Teams**
- **Training**

❌ When NOT to Use

- **Production environments** — resource contention is a real risk
- **High job concurrency** — execution will compete with control overhead
- **Security-sensitive setups** — separation of planes is best practice

SCALING

When to Add More Execution Nodes

🚦 Warning Signs

Symptom	Meaning
Jobs stuck in Pending	No execution capacity
Long queue times	Execution nodes overloaded
High CPU / RAM usage	Node saturation
Parallel jobs failing	Insufficient forks/resources

📏 Scaling Rule of Thumb

1 Control Node can manage many execution nodes. Add execution nodes when:

- You need more **parallel job** capacity
- Jobs run **longer or touch more hosts**
- Automation scope grows across teams

> Scale **execution nodes horizontally** — don't overload the ones you have.

CAPACITY PLANNING

Capacity Planning Basics

Start with this simple formula when scoping a new AAP deployment:

> **Execution Nodes Needed ≈ Concurrent Jobs × Job Complexity**

1	2
Lab 1 Hybrid Node — combines control and execution for testing and learning	**Small Prod** 1 Control + 2 Execution Nodes — suitable for limited team automation
3	**4**
Medium Prod 2 Control (HA) + 4–6 Execution Nodes — handles moderate parallelism	**Large Enterprise** HA Control + Many Execution + Hop Nodes — full-scale distributed automation

TCP/27199

2.2 — Node Types Explained

A deeper look at the control plane, execution plane, hop nodes, and instance groups — and how they work together in a production AAP deployment.

CONTROL PLANE

The Control Plane — Where Decisions Are Made

The control plane handles:

- **Job scheduling** — deciding when and where jobs run
- **Inventory resolution** — translating hosts and groups
- **Credential injection** — securely passing secrets to jobs
- **Instance group selection** — routing jobs to the right nodes
- **Job routing** — traffic management across the platform

⚠️ **Key Point:** The control plane **never touches managed hosts directly** — that responsibility lives entirely in the execution plane.

EXECUTION PLANE

The Execution Plane — Where Automation Runs

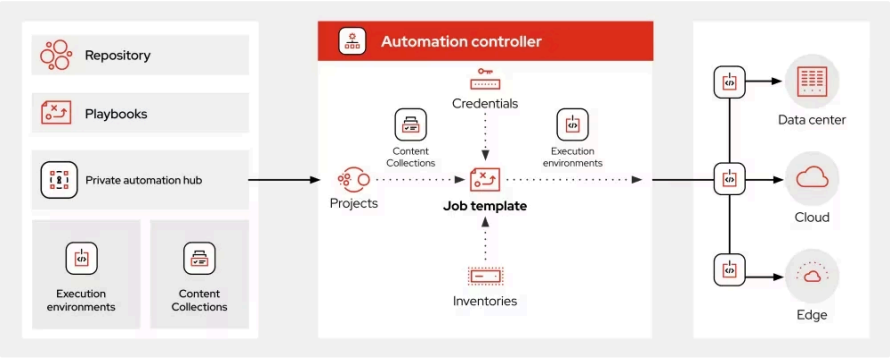
What runs here:

Playbook Execution

Module Execution

SSH / WinRM / API

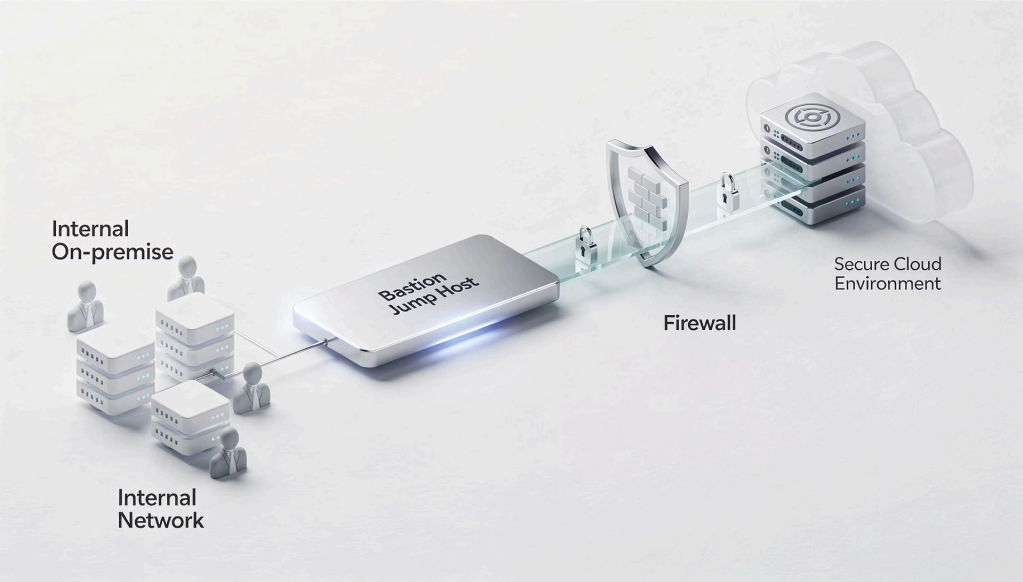
Task Retries & Forks



The execution plane is where your **actual automation work** happens — playbooks run, hosts are configured, and tasks are completed.

NODE TYPES

Hop Nodes — Connecting Isolated Networks



Why Hop Nodes Exist

Many enterprise environments present network barriers that prevent direct automation:

- Systems behind **firewalls** with no inbound access
- Hosts in **DMZs** or segmented network zones
- **Air-gapped environments** with strict isolation

What a Hop Node Does

It acts as a **secure relay** — forwarding automation traffic from the execution plane into otherwise unreachable zones without opening direct routes.

> 🔐 **Analogy:** Hop Node = Bastion / Jump Host — a controlled entry point into a restricted zone.

NODE TYPES

Instance Groups — Organizing Nodes by Purpose

Instance groups are **logical containers** for execution nodes. Rather than treating all execution capacity as one pool, instance groups let you organize nodes by team, environment, or geographic region.

Targeted Execution

Assign specific jobs to specific instance groups — e.g., run production jobs only on production-hardened nodes

Environment Isolation

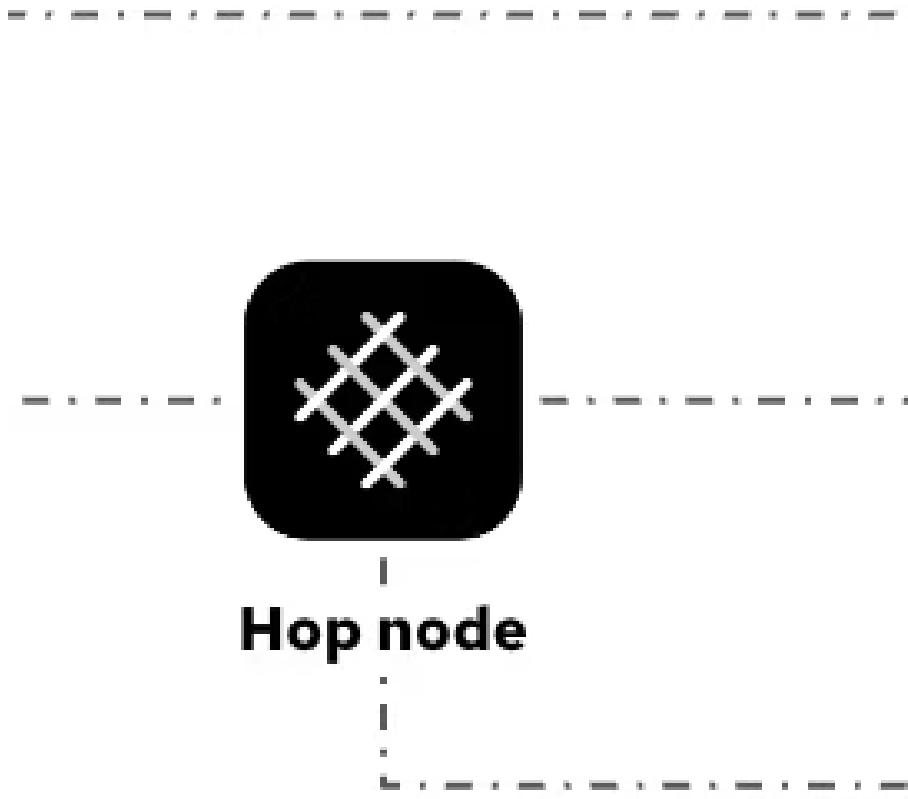
Separate dev, staging, and prod workloads into their own execution pools to prevent cross-environment interference

Controlled Capacity

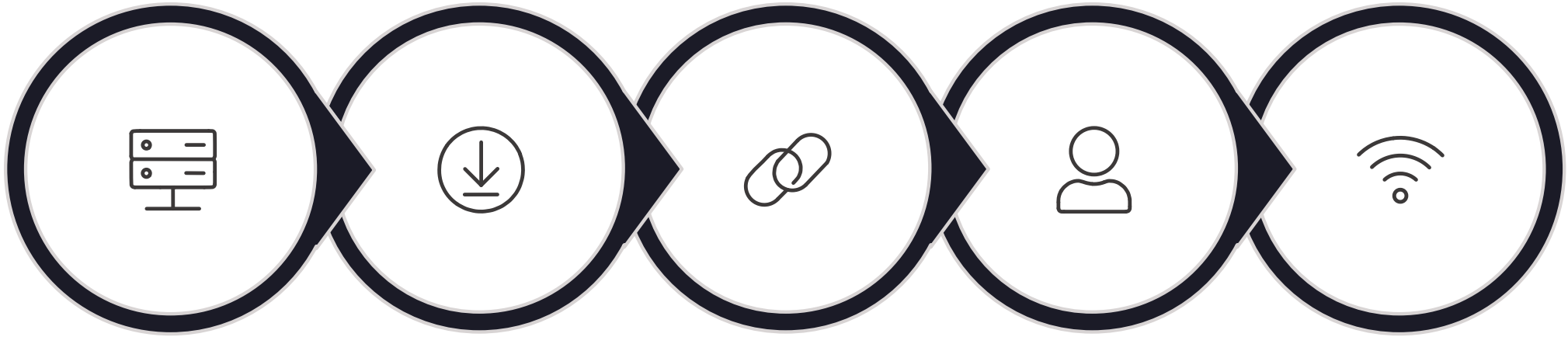
Limit how many jobs can run in a given group, protecting critical infrastructure from being overloaded

2.3 — Managing Nodes

How to add, monitor, scale, and safely decommission execution nodes in a production AAP environment.



Adding New Execution Nodes



Provision VM → **Install Packages** → **Register Node** → **Assign Group** → **Validate Connectivity**

Each step must be completed in order. Registration links the node to the controller's mesh network; validation confirms the node is ready to receive jobs before any live workloads are assigned.

MANAGING NODES

Monitoring Node Health & Status

AAP Continuously Checks:

- **Heartbeat** — is the node responding?
- **CPU / Memory** — is the node under resource pressure?
- **Job execution capability** — can it accept new jobs?
- **Connectivity to controller** — is the mesh link healthy?

Node Statuses You'll See:

✅ Ready

Healthy and accepting jobs

🔴 Offline

Node unreachable or stopped

⚠️ Unavailable

Temporarily cannot run jobs

🟡 Degraded

Running but with reduced capacity

MANAGING NODES

Node Capacity & Job Distribution

How the Scheduler Distributes Jobs

Every execution node is assigned a **capacity score** derived from its available CPU and memory. The AAP scheduler uses this score to make intelligent placement decisions:

- Jobs are **queued** at the instance group level
- The scheduler **assigns** queued jobs to nodes with sufficient capacity
- Jobs **execute** only on nodes that can handle the fork count

> A node with a higher capacity score will absorb more jobs — right-size your VMs to maximize utilization.

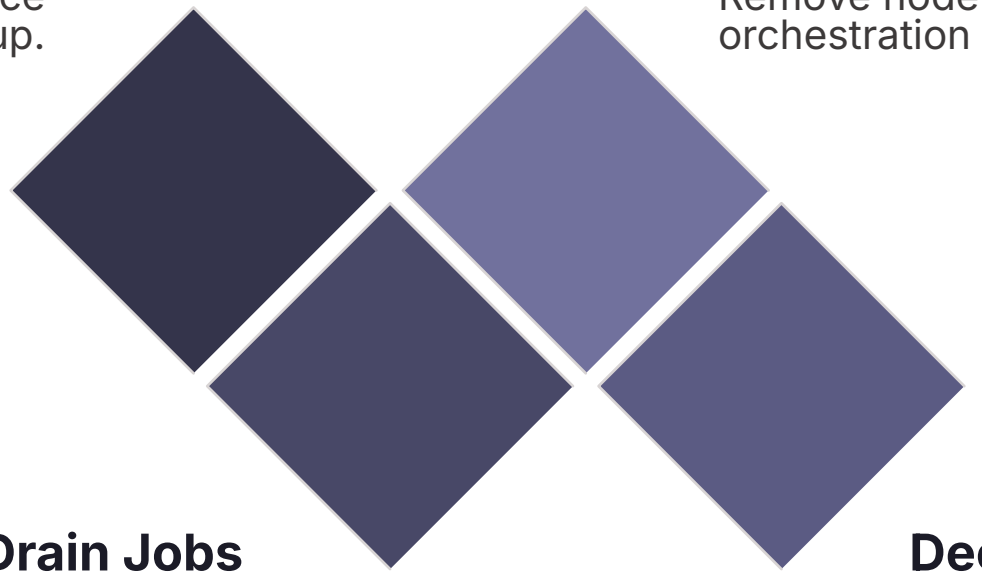
Removing & Decommissioning Nodes Safely

Remove From Group

Take node out of instance group.

Deregister Controller

Remove node from orchestration control plane.



Drain Jobs

Evict running tasks before removal.

Decommission VM

Power off and delete the virtual machine.

⚠️ **Critical:** Never delete a node while jobs are actively running on it. Always drain first — abrupt removal causes job failures and can leave automation in an unknown state.

TROUBLESHOOTING

Basic Node Connectivity Troubleshooting

Common Issues

Firewall ports blocked

Receptor mesh requires specific ports open between nodes

DNS resolution failure

Nodes must resolve each other by hostname

Clock skew (NTP)

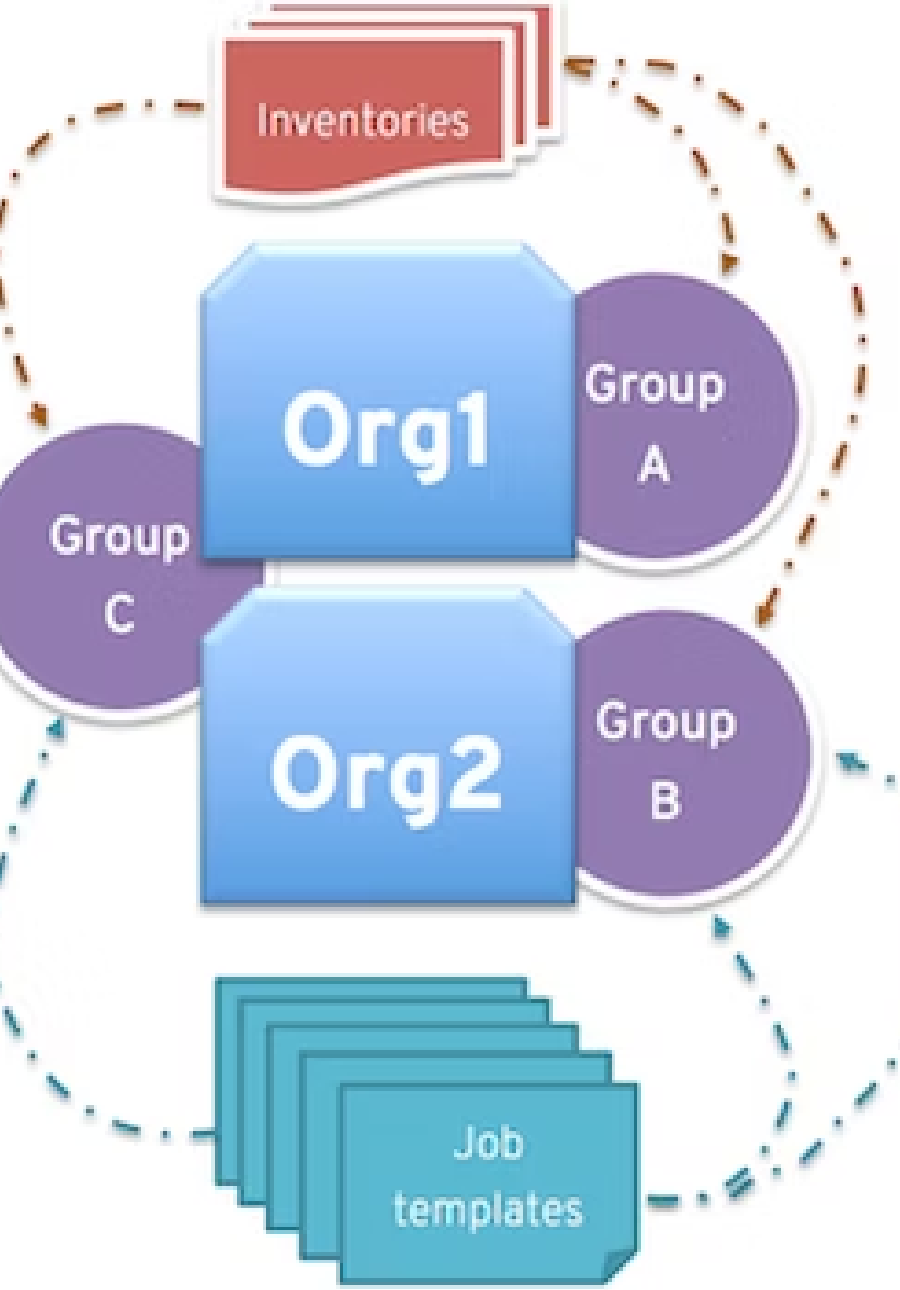
Time drift causes TLS and auth failures

TLS certificate mismatch

Mismatched certs block secure mesh communication

Basic Diagnostic Checks

Check	Tool / Command
Network reachability	ping / curl to node port
Time synchronization	timedatectl / chronyc
Service status	systemctl status receptor
Logs	journalctl -u receptor -f



CHAPTER

2.4 — Instance Groups Fundamentals

Instance groups are the primary mechanism for **organizing, targeting, and isolating automation workloads** across your AAP infrastructure. Understanding them is essential before scaling any production deployment.

Group by Environment	Group by Team
Dev, staging, and prod each get their own node pool	Network, security, and app teams use dedicated capacity
Group by Region	
Route jobs to geographically appropriate execution nodes	